1. Linear Regression

```
# Step 1: Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Step 2: Create dataset
data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8],
    "Scores": [20, 25, 35, 40, 50, 55, 65, 75]
}

df = pd.DataFrame(data)
print(df)

# Step 3: Separate input and output
X = df[["Hours"]]
y = df["Scores"]

# Step 4: Create and train model
model = LinearRegression()
model.fit(X, y)

# Step 5: Predict scores
y_pred = model.predict(X)

# Step 6: Calculate MSE and R2 score
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

# Step 7: Calculate accuracy (within 5 marks)
accuracy = (abs(y - y_pred) <= 5).mean() * 100

# Step 8: Print results
print("MSE:", round(mse, 2))
print("R2 Score:", round(r2, 4))
print("Accuracy:", round(accuracy, 2), "%")
```

```
# Step 9: Plot actual data
plt.scatter(X, y, color="blue", label="Actual Data")

# Step 10: Plot regression line
plt.plot(X, y_pred, color="red", label="Regression Line")

# Step 11: Add labels and show graph
plt.xlabel("Hours Studied")
plt.ylabel("Scores")
plt.title("Study Hours vs Score")
plt.legend()
plt.show()
```

2. Simplest Logistic Regression Code

```
# Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# Step 2: Create dataset
data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Pass":  [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)
print(df)

# Step 3: Separate input and output
X = df[["Hours"]]
y = df["Pass"]
```

```
# Step 4: Create and train model
model = LogisticRegression()
model.fit(X, y)

# Step 5: Predict output
y_pred = model.predict(X)

# Step 6: Calculate accuracy
print("Accuracy:", accuracy_score(y, y_pred) * 100, "%")

# Step 7: Display confusion matrix
cm = confusion_matrix(y, y_pred)
print("Confusion Matrix:")
print(cm)

ConfusionMatrixDisplay(confusion_matrix=cm).plot()
plt.show()

# Step 8: Calculate probability for sigmoid curve
x = np.linspace(1, 10, 100).reshape(-1, 1)
prob = model.predict_proba(x)[:, 1]

# Step 9: Plot actual data and sigmoid curve
plt.scatter(X, y, color="blue", label="Actual Data")
plt.plot(x, prob, color="red", label="Sigmoid Curve")

plt.xlabel("Hours Studied")
plt.ylabel("Probability of Passing")
plt.title("Logistic Regression")
plt.legend()
plt.grid()
plt.show()
```

3. Decision Tree Algo

```
# Step 1: Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay, classification_report

# Step 2: Create dataset
data = {

"Age":[18,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,4
1,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,58,60],

"Salary":[18000,22000,24000,26000,28000,30000,32000,35000,37000,39000,
42000,45000,47000,50000,52000,55000,57000,60000,62000,65000,67000,7
0000,72000,74000,76000,78000,80000,82000,84000,86000,88000,90000,92
000,94000,96000,98000,100000,102000,105000,110000],

"Purchased":[0,0,0,0,0,0,0,0,1,0,1,0,1,0,1,1,0,1,1,0,1,1,0,1,1,1,0,1,1,1,0,1,1,1,
1,1,1,1,1,1]
}

df = pd.DataFrame(data)
print(df)

# Step 3: Separate input and output
X = df[["Age", "Salary"]]
y = df["Purchased"]

# Step 4: Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Step 5: Create and train Decision Tree
model = DecisionTreeClassifier(max_depth=6, random_state=42)
model.fit(X_train, y_train)

# Step 6: Predict results
y_pred = model.predict(X_test)

# Step 7: Calculate accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Step 8: Display confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No", "Yes"]
).plot()

plt.show()

# Step 9: Display classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Step 10: Plot Decision Tree
plt.figure(figsize=(12, 8))

plot_tree(
    model,
    feature_names=["Age", "Salary"],
    class_names=["No", "Yes"],
    filled=True,
    rounded=True
)

plt.title("Decision Tree")
plt.show()
```

4. Ensemble Model

```
# Step 1: Import libraries
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
GradientBoostingClassifier
from sklearn.metrics import accuracy_score

# Step 2: Create dataset
X = [
    [1,45,30], [2,50,35], [2,65,40], [3,55,42], [3,70,45],
    [4,60,48], [4,75,50], [5,55,52], [5,68,55], [5,80,58],
    [6,60,60], [6,72,62], [6,85,65], [7,65,67], [7,78,69],
    [7,90,72], [8,70,74], [8,82,76], [8,88,78], [9,75,80],
    [9,85,82], [9,95,85], [10,78,86], [10,88,88], [10,95,90],
    [11,80,92], [11,90,94], [12,85,95], [12,92,96], [13,95,98]
]

y = [0,0,0,0,0,0,1,0,1,1,0,1,1,1,1,1,1,1,1,
     1,1,1,1,1,1,1,1,1,1]

# Step 3: Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=7, stratify=y
)

# Step 4: Create four models
models = {
    "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=7),
    "Random Forest": RandomForestClassifier(n_estimators=50, max_depth=4,
random_state=7),
    "AdaBoost": AdaBoostClassifier(n_estimators=30, learning_rate=0.5,
random_state=7),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=50,
max_depth=2, random_state=7)
}

# Step 5: Train, predict and calculate accuracy
```

```
results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy

    print("\n", name)
    print("Actual:", y_test)
    print("Predicted:", y_pred)
    print("Accuracy:", round(accuracy * 100, 2), "%")

# Step 6: Compare all models
print("\nMODEL COMPARISON")

for name, accuracy in results.items():
    print(name, ":", round(accuracy * 100, 2), "%")

# Step 7: Find the best model
best_model = max(results, key=results.get)
print("Best Model:", best_model)
print("Best Accuracy:", round(results[best_model] * 100, 2), "%")

# Step 8: Plot accuracy comparison
plt.bar(results.keys(), [v * 100 for v in results.values()])
plt.xlabel("Models")
plt.ylabel("Accuracy (%)")
plt.title("Model Comparison")
plt.ylim(0, 100)
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

5. SVM

```
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# Step 2: Create dataset
X = np.array([
    [120,6.0], [125,6.2], [130,6.3], [135,6.5],
    [140,6.6], [145,6.8], [150,7.0], [155,7.1],
    [160,7.2], [165,7.4], [170,7.5], [175,7.7],
    [180,7.8], [185,8.0], [190,8.2], [195,8.3]
])

# 0 = Apple, 1 = Orange
y = np.array([0,0,0,0,0,0,0,0,1,1,1,1,1,1,1,1])

# Step 3: Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Step 4: Scale data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Train SVM model
model = SVC(kernel="linear")
model.fit(X_train, y_train)

# Step 6: Predict and calculate accuracy
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

```
# Step 7: Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Apple", "Orange"]
).plot()
plt.show()

# Step 8: Display support vectors
print("Support Vectors:")
print(model.support_vectors_)
print("Number of Support Vectors:", len(model.support_vectors_))

# Step 9: Plot Apple and Orange
plt.scatter(X_train[y_train == 0, 0], X_train[y_train == 0, 1],
            color="blue", label="Apple")

plt.scatter(X_train[y_train == 1, 0], X_train[y_train == 1, 1],
            color="red", label="Orange")

# Highlight support vectors
plt.scatter(model.support_vectors_[:, 0],
            model.support_vectors_[:, 1],
            s=150, facecolors="none", edgecolors="black",
            label="Support Vectors")

# Step 10: Calculate hyperplane and margins
w = model.coef_[0]
b = model.intercept_[0]
x = np.linspace(-2, 2, 100)

y_line = -(w[0] * x + b) / w[1]
y_upper = -(w[0] * x + b - 1) / w[1]
y_lower = -(w[0] * x + b + 1) / w[1]

# Step 11: Plot hyperplane and margins
plt.plot(x, y_line, color="green", label="Hyperplane")
```

```
plt.plot(x, y_upper, "g--", label="Margins")
plt.plot(x, y_lower, "g--")

plt.xlabel("Weight (Scaled)")
plt.ylabel("Size (Scaled)")
plt.title("SVM: Apple vs Orange")
plt.legend()
plt.grid()
plt.show()
```

6. DBSCAN algo

```
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN

# Step 2: Create dataset
X = np.array([
    [1, 2], [2, 2], [2, 3],
    [8, 7], [8, 8], [9, 8],
    [20, 20]
])

# Step 3: Create DBSCAN model
model = DBSCAN(eps=2, min_samples=2)

# Step 4: Find clusters
labels = model.fit_predict(X)

# Step 5: Print cluster labels
print("Cluster Labels:", labels)

# Step 6: Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("DBSCAN Clustering")
plt.show()
```

7. PCA Algo

```
# Step 1: Import libraries
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Step 2: Take input
rows = int(input("Enter number of data points: "))
cols = int(input("Enter number of features: "))

# Step 3: Enter dataset values
X = []

for i in range(rows):
    row = []
    for j in range(cols):
        value = float(input("Enter value: "))
        row.append(value)
    X.append(row)

X = np.array(X)

print("Original Dataset:")
print(X)

# Step 4: Scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 5: Take number of components
n = int(input("Enter number of principal components: "))

# Step 6: Apply PCA
pca = PCA(n_components=n)
X_pca = pca.fit_transform(X_scaled)

# Step 7: Display results
print("Transformed Dataset:")
print(X_pca)
```

```
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("Total Variance Retained:")
print(sum(pca.explained_variance_ratio_))

print("Principal Components:")
print(pca.components_)
```